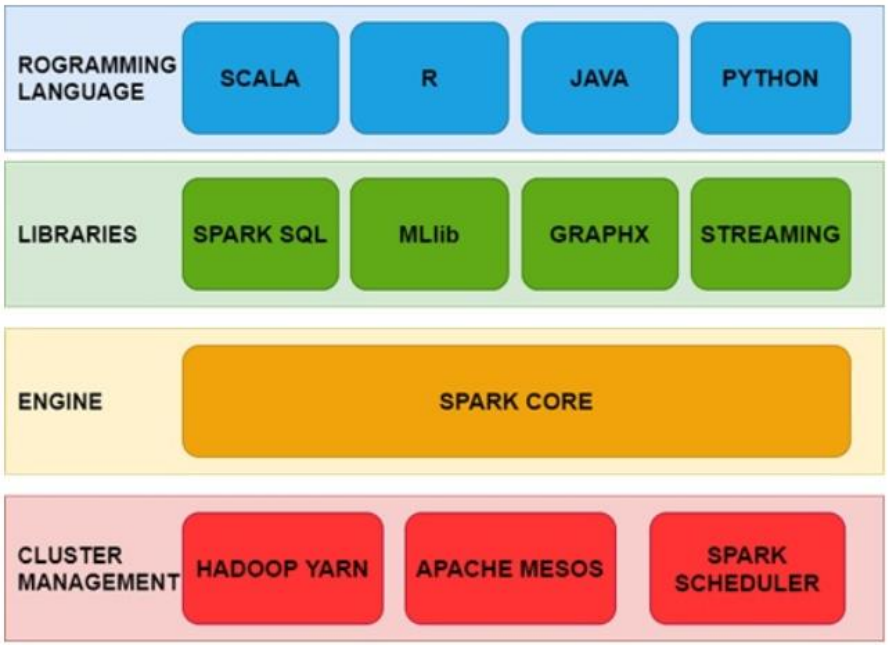
# MODUL 12 – SPARK

### 1.1. Deskripsi Singkat

Apache Spark adalah kerangka kerja pemrosesan paralel berbasis *open source* yang mendukung pemrosesan dalam memori untuk meningkatkan performa aplikasi dalam menganalisis big data. Solusi big data dirancang untuk menangani data yang terlalu besar atau kompleks untuk database tradisional. Spark memproses sejumlah besar data dalam memori, yang jauh lebih cepat daripada alternatif berbasis disk.




Fitur yang ada pada Apache Spark:

- ✓ Performa lebih cepat dibandingkan framework pemrosesan data tradisional.
- ✓ Mudah digunakan, aplikasi pengolahan data yang dibangun dengan Spark dapat dituliskan dalam bahasa pemrograman Python, R, Java, dan Scala.
- ✓ Dilengkapi dengan SQL Library, Streaming, dan Graph Analysis yang memudahkan proses pengolahan dan alnalisis data.

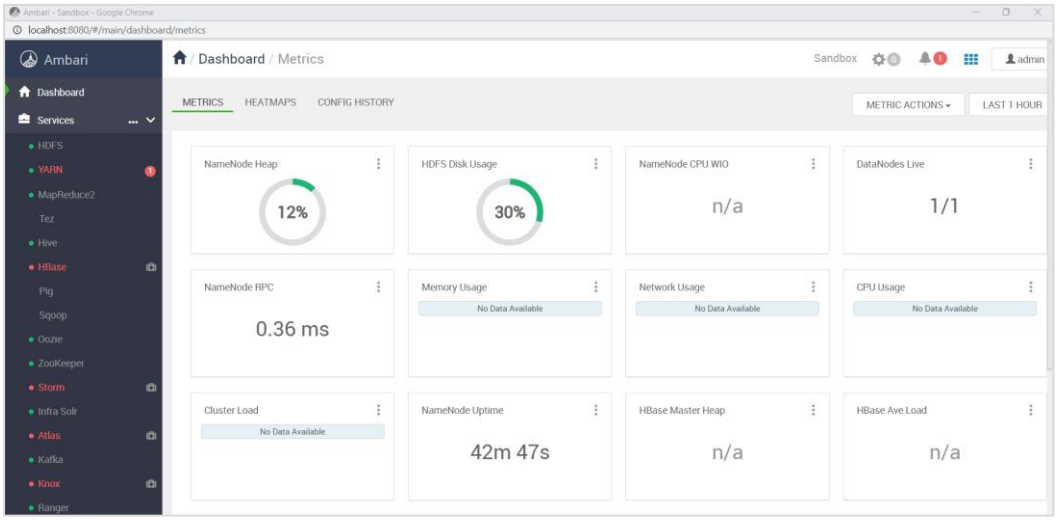
### 1.2. Tujuan Praktikum

Setelah praktikum pada modul 12 ini, diharapkan mahasiswa mempunyai kompetensi dalam menggunakan Spark dalam pemrosesan data.

### 1.3. Material Praktikum

Persyaratan yang dibutuhkan untuk melakukan praktikum yaitu :

1. HDP yang telah terinstal pada VirtualBox.
2. Ambari dapat diakses

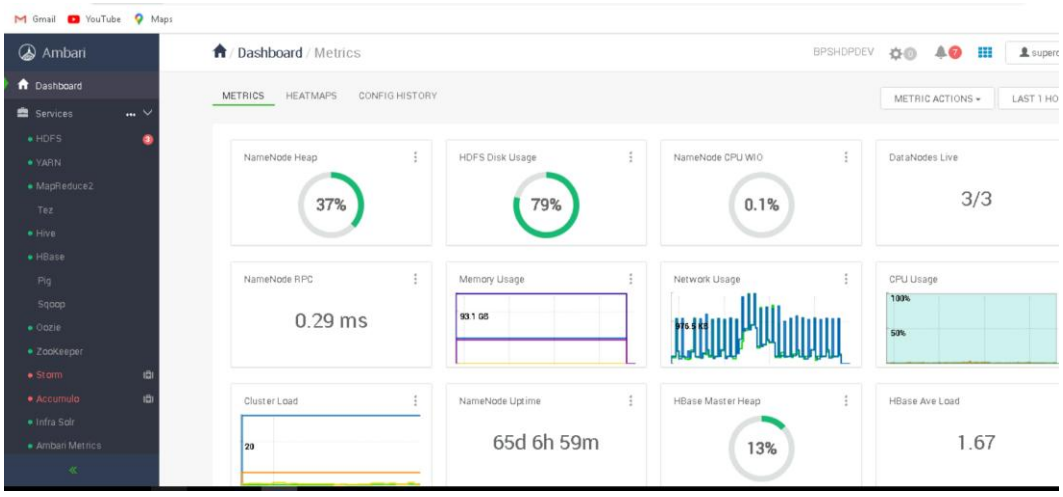



3. Service HDFS, Oozie, Zookeeper, YARN, Spark dalam keadaan hidup (*on*) dan dapat digunakan.

### 1.4. Kegiatan Praktikum

#### A. Spark SQL

1. Akses Ambari dengan menggunakan akun yang telah dibuat sebelumnya untuk memastikan service yang dibutuhkan dalam keadaan hidup dan siap digunakan, terutama HDFS, Oozie, Zookeeper, YARN, Hive, Spark2.

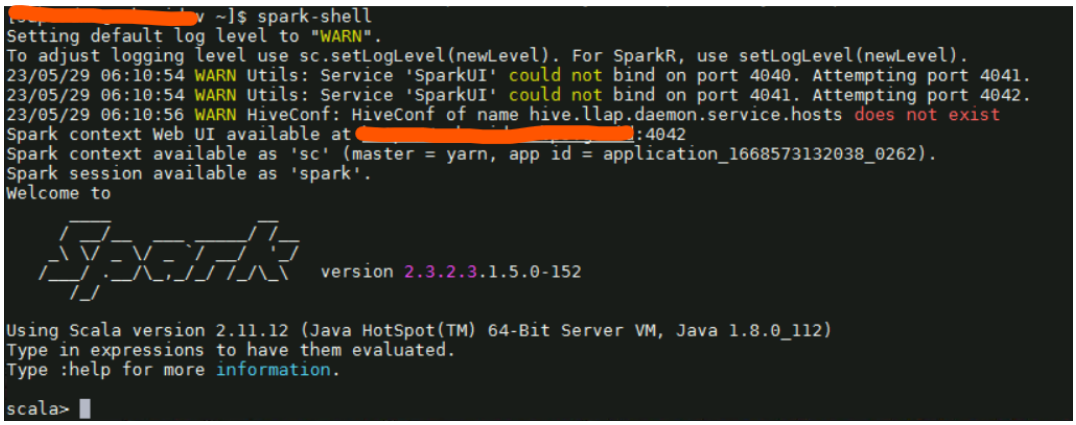


2. Untuk memastikan Spark telah terinstal dan siap digunakan, lakukan langkah-langkah berikut.
    1) Akses http://localhost:4200 pada browser.
    2) Lakukan login terlebih dahulu.

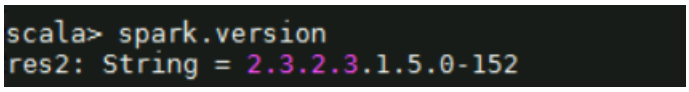
       Username: **root**

       Existing Password: **Tp4stis**
    3) Jalankan command:

       a) `spark-shell`




       b) `spark.version`




3. Secara default, spark-shell menyediakan objek **spark** (SparkSession) dan **sc** (SparkContext) yang siap untuk digunakan.

```
scala> spark
scala> sc
```




4. Mengecek current spark session

```
scala> import org.apache.spark.sql.SparkSession
scala> val spark2 = SparkSession.builder().getOrCreate()
scala> print(spark2)
```

```
scala> import org.apache.spark.sql.SparkSession
import org.apache.spark.sql.SparkSession

scala> val spark2 = SparkSession.builder().getOrCreate()
spark2: org.apache.spark.sql.SparkSession = org.apache.spark.sql.SparkSession@2100b770

scala> print(spark2)
org.apache.spark.sql.SparkSession@2100b770
scala>
```

5. Create RDD

   Untuk membuat RDD, jalankan command berikut.

   a) Import **org.apache.spark.sql.SparkSession** untuk dapat mengakses SparkSession class.

```
scala> import org.apache.spark.sql.SparkSession
```

```
scala> import org.apache.spark.sql.SparkSession
import org.apache.spark.sql.SparkSession
```

   b) Input data dengan menggunakan **Seq**:

```
scala> val dataSeq = Seq(("Java", 2000), ("Phyton", 100000),
("Scala", 3000))
```

```
scala> val dataSeq = Seq(("Java", 20000), ("Python", 100000), ("Scala", 3000))
dataSeq: Seq[(String, Int)] = List((Java,20000), (Python,100000), (Scala,3000))
```

**Seq** merupakan built-in Scala collection type yang merepresentasikan sebuah urutan elemen-elemen. Seq nantinya akan digunakan dalam operasi Spark, seperti membuat RDD (Resilient Distributed Datasets) atau DataFrames. Untuk membuat RDD dari Seq gunakan perintah:

```
scala> val rdd = spark.sparkContext.parallelize(dataSeq)
```

```
scala> val rdd = spark.sparkContext.parallelize(dataSeq)
rdd: org.apache.spark.rdd.RDD[(String, Int)] = ParallelCollectionRDD[0] at parallelize at <console>:28
```

   c) Menampilkan data

      Untuk menampilkan data yang telah diinput ke dalam RDD, gunakan **collect()** yang akan mengakses data dan mengirimkan ke driver serta melakukan perulangan hingga semua konten data yang ada di RDD tampil.

```
scala> val data = rdd.collect()
```

```
scala> val data = rdd.collect()
data: Array[(String, Int)] = Array((Java,20000), (Python,100000), (Scala,3000))
```

```
scala> data.foreach(println)
```

```
scala> data.foreach(println)
(Java,20000)
(Python,100000)
(Scala,3000)
```

```
scala> data.foreach(f=>println(f._1+","+f._2))
```

```
scala> data.foreach(f=>println(f._1 +","+f._2))
Java,20000
Python,100000
Scala,3000
```

6. RDD Operation

Seperti yang telah dijelaskan, terdapat dua operasi yang dapat dilakukan, yaitu *transformations* dan *actions*.

Contoh di bawah menggunakan Seq bukan RDD. Operasi-operasi *transformation* yang sama juga dapat diterapkan pada RDD tetapi hasilnya tidak dapat ditampilkan secara langsung. Jika menggunakan RDD seperti:

```
scala> val filter = rdd.filter(a=> a._1.startsWith("J"))
```

Maka untuk menampilkan perlu menggunakan *action* **collect()** seperti dicontohkan pada langkah sebelumnya.

Beberapa contoh *transformation* yaitu sebagai berikut.

```
scala> val filter = dataSeq.filter(a=> a._1.startsWith("J"))
```

```
scala> val filter = dataSeq.filter(a=> a._1.startsWith("J"))
filter: Seq[(String, Int)] = List((Java,20000))
```

```
scala> val filter = dataSeq.filter(a=> a._1.startsWith("P"))
```

```
scala> val filter = dataSeq.filter(a=> a._1.startsWith("P"))
filter: Seq[(String, Int)] = List((Python,100000))
```

```
scala> val filter = dataSeq.filter(a=> a._1.contains("P"))
```

```
scala> val filter = dataSeq.filter(a=> a._1.contains("P"))
filter: Seq[(String, Int)] = List((Python,100000))
```

```
scala> val filter = dataSeq.filter(a=> a._1.contains("a"))
```

```
scala> val filter = dataSeq.filter(a=> a._1.contains("a"))
filter: Seq[(String, Int)] = List((Java,20000), (Scala,3000))
```

```
scala> val filter = dataSeq.filter(a=> a._1.matches("a"))
```

```
scala> val filter = dataSeq.filter(a=> a._1.matches("a"))
filter: Seq[(String, Int)] = List()
```

```
scala> val filter = dataSeq.filter(a=> a._1.matches(".*a.*"))
```

```
scala> val filter = dataSeq.filter(a=> a._1.matches(".*a.*"))
filter: Seq[(String, Int)] = List((Java,20000), (Scala,3000))
```

```
scala> val filter = dataSeq.filter(a=> a._1.length>5)
```

```
scala> val filter = dataSeq.filter(a=> a._1.length>5)
filter: Seq[(String, Int)] = List((Python,100000))
```

```
scala> val filter = dataSeq.filter(a=> a._2>3000)
```

```
scala> val filter = dataSeq.filter(a=> a._2>3000)
filter: Seq[(String, Int)] = List((Java,20000), (Python,100000))
```

```
scala> val filter = dataSeq.sortBy(a=> a._2)
```

```
scala> val sort = dataSeq.sortBy(a=>a._2)
sort: Seq[(String, Int)] = List((Scala,3000), (Java,20000), (Python,100000))
```

7. Create DataFrame

   Selain mengakses data dengan menggunakan RDD, Spark juga memungkinkan untuk mengakses data dengan menggunakan dataframe. Sebelum membuat dataframe, kita dapat membuat schema tabel terlebih dahulu. Selanjutnya, schema tersebut akan dimapping dengan data yang telah load sebelumnya ke RDD dan dibentuk menjadi dataframe.

```
scala> val columns = Seq("programming_language","total_user")
```

```
scala> val collumns = Seq("programming_language","total_user")
collumns: Seq[String] = List(programming_language, total_user)
```

```
scala> val df =
spark.createDataFrame(dataSeq).toDF(columns:_*)
```

```
scala> val df = spark.createDataFrame(dataSeq).toDF(collumns:_*)
df: org.apache.spark.sql.DataFrame = [programming_language: string, total_user: int]
```

Untuk menampilkan data, dapat menggunakan fungsi **show()**.

```
scala> df.show()
```

```
scala> df.show()
+--------------------+----------+
|programming_language|total_user|
+--------------------+----------+
|                Java|     20000|
|              Python|    100000|
|               Scala|      3000|
+--------------------+----------+
```

Untuk menampilkan schema, dapat menggunakan fungsi **printSchema()**.

```
scala> df.printSchema()
```

```
scala> df.printSchema()
root
 |-- programming_language: string (nullable = true)
 |-- total_user: integer (nullable = false)
```

8. DataFrame Operation

   Seperti halnya RDD, data yang dibentuk menjadi dataframe juga dapat dilakukan transformasi, seperti select kolom, modifikasi konten kolom, agregasi.

```
scala> df.select("programming_language").show()
```

```
scala> df.select("programming_language").show()
+--------------------+
|programming_language|
+--------------------+
|                Java|
|              Python|
|               Scala|
+--------------------+
```

```
scala> val df2 = df.withColumn("total_user",
col("total_user")*100)
scala> df2.show()
```

```
scala> val df2 =df.withColumn("total_user",col("total_user")*100)
df2: org.apache.spark.sql.DataFrame = [programming_language: string, total_user: int]

scala> df2.show()
+--------------------+----------+
|programming_language|total_user|
+--------------------+----------+
|                Java|   2000000|
|              Python|  10000000|
|               Scala|    300000|
+--------------------+----------+
```

```
scala> df.groupBy("programming_language").count.show()
```

```
scala> df.groupBy("programming_language").count.show()
+--------------------+-----+
|programming_language|count|
+--------------------+-----+
|               Scala|    1|
|              Python|    1|
|                Java|    1|
+--------------------+-----+
```

9. Save to Table

a) Akses data dari HDFS

Untuk tahap ini, dapat menggunakan file yang telah di import ke dalam hdfs dengan menggunakan Sqoop pada praktikum 8.

```
[root@sandbox-hdp ~]# hdfs dfs -ls
Found 13 items
drwx------   - root hdfs          0 2023-03-19 12:00 .Trash
drwxr-xr-x   - root hdfs          0 2023-05-15 01:17 .sparkStaging
drwx------   - root hdfs          0 2023-03-19 09:24 .staging
drwxr-xr-x   - root hdfs          0 2023-03-19 07:25 _sqoop
drwxr-xr-x   - root hdfs          0 2023-02-14 18:44 geo_data
drwxr-xr-x   - root hdfs          0 2023-03-19 03:24 input
drwxr-xr-x   - root hdfs          0 2023-03-19 03:40 output
drwxr-xr-x   - root hdfs          0 2023-03-19 04:06 sdata
drwxr-xr-x   - root hdfs          0 2023-03-19 07:23 sqoopdata
drwxr-xr-x   - root hdfs          0 2023-03-19 07:25 sqoopdata1
drwxr-xr-x   - root hdfs          0 2023-03-19 08:02 sqoopdata2
drwxr-xr-x   - root hdfs          0 2023-03-19 09:24 sqoopdata3
drwxr-xr-x   - root hdfs          0 2023-02-14 20:13 tdata
[root@sandbox-hdp ~]# hdfs dfs -cat sqoopdata/*
1,John,5000,IT,Developer
2,Chris,6000,Sales,Manager
3,Jamie,7000,Support,Director
4,Hening,4000,IT,Developer
```

Path data di hdfs dapat disesuaikan sesuai dengan praktikum 8.

```
scala> val data_hdfs =
spark.read.csv("/user/root/sqoopdata/")
scala> data_hdfs.show()
```

```
scala> val data_hdfs = spark.read.csv("/user/superdms/UserLatihan/Import2/")
data_hdfs: org.apache.spark.sql.DataFrame = [_c0: string, _c1: string ... 3 more fields]

scala> data_hdfs.show()
+---+-------+----+-------+---------+
|_c0|    _c1| _c2|    _c3|      _c4|
+---+-------+----+-------+---------+
|  1|   John|5000|     IT|Developer|
|  2|  Chris|6000|  Sales|  Manager|
|  3|  Jamie|7000|Support| Director|
|  4|Henning|4000|     IT|Developer|
+---+-------+----+-------+---------+
```

b) Membuat skema dan melakukan mapping data dengan skema saat melakukan load data dari hdfs.

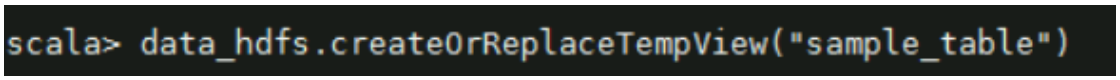
```
scala> import org.apache.spark.sql.types._
scala> val simpleSchema = StructType(Seq(StructField("id",
StringType), StructField("name", StringType),
StructField("salary", IntegerType), StructField("department",
StringType), StructField("role", StringType)))
scala> val data_hdfs =
spark.read.schema(simpleSchema).csv("/user/root/sqoopdata/")
scala> data_hdfs.show()
```

```
scala> val data_hdfs = spark.read.schema(simpleSchema).csv("/user/superdms/UserLatihan/Import2/")
data_hdfs: org.apache.spark.sql.DataFrame = [id: string, name: string ... 3 more fields]

scala> data_hdfs.show()
+---+-------+------+----------+---------+
| id|   name|salary|department|     role|
+---+-------+------+----------+---------+
|  1|   John|  5000|        IT|Developer|
|  2|  Chris|  6000|     Sales|  Manager|
|  3|  Jamie|  7000|   Support| Director|
|  4|Henning|  4000|        IT|Developer|
+---+-------+------+----------+---------+
```

c) Simpan hasil ke tabel sementara

```
scala> data_hdfs.createOrReplaceTempView("sample_table")
```




d) Lakukan operasi berikut terhadap data yang tersimpan pada tabel sementara

```
scala> val data_hdfs2 = spark.sql("SELECT * FROM
sample_table")
scala> data_hdfs2.show()
```

```
scala> val data_hdfs2 = spark.sql("SELECT * FROM sample_table")
data_hdfs2: org.apache.spark.sql.DataFrame = [id: string, name: string ... 3 more fields]

scala> data_hdfs.show()
+---+-------+------+----------+---------+
| id|   name|salary|department|     role|
+---+-------+------+----------+---------+
|  1|   John|  5000|        IT|Developer|
|  2|  Chris|  6000|     Sales|  Manager|
|  3|  Jamie|  7000|   Support| Director|
|  4|Henning|  4000|        IT|Developer|
+---+-------+------+----------+---------+
```

```
scala> val salary_more_than_5000 = spark.sql("SELECT * FROM
sample_table where salary > 5000")
scala> salary_more_than_5000.show()
```

```
scala> val salary_more_than_5000 = spark.sql("SELECT * FROM sample_table where salary > 5000")
salary_more_than_5000: org.apache.spark.sql.DataFrame = [id: string, name: string ... 3 more fields]
```

```
scala> salary_more_than_5000.show()
+---+-----+------+----------+--------+
| id| name|salary|department|    role|
+---+-----+------+----------+--------+
|  2|Chris|  6000|     Sales| Manager|
|  3|Jamie|  7000|   Support|Director|
+---+-----+------+----------+--------+
```

e) Simpan ke tabel external hive:

```
scala>
spark.table("sample_table").write.saveAsTable("sample_hive_ta
ble")
```

```
scala> spark.table("sample_table").write.saveAsTable("sample_hive_table")
```

Untuk mengecek tabel hive yang telah dibuat, dapat dilakukan akses ke database: **'default'** dengan nama tabel **'sample_hive_table'**.

```
scala> val read_salary2 = spark.sql("SELECT * FROM
sample_hive_table")
scala> read_salary2.show()
```

```
scala> val read_salary2  = spark.sql("SELECT * FROM sample_hive_table")
read_salary2: org.apache.spark.sql.DataFrame = [id: string, name: string ... 3 more fields]

scala> read_salary2.show()
+---+-------+------+----------+---------+
| id|   name|salary|department|     role|
+---+-------+------+----------+---------+
|  1|   John|  5000|        IT|Developer|
|  2|  Chris|  6000|     Sales|  Manager|
|  3|  Jamie|  7000|   Support| Director|
|  4|Henning|  4000|        IT|Developer|
+---+-------+------+----------+---------+
```

10. Simpan ke format data lain, seperti orc

```
scala> salary_more_than_5000.write.orc("/user/root/
salary_more_than_5000/")
```

```
scala> salary_more_than_5000.write.orc("/user/superdms/UserLatihan/salary_more_than_5000/")
```

```
scala> val read_salary = spark.read.orc("/user/root/
salary_more_than_5000/")
scala> read_salary.show()
```

```
scala> val read_salary  = spark.read.orc("/user/superdms/UserLatihan/salary_more_than_5000/")
read_salary: org.apache.spark.sql.DataFrame = [id: string, name: string ... 3 more fields]

scala> read_salary.show()
+---+-----+------+----------+--------+
| id| name|salary|department|    role|
+---+-----+------+----------+--------+
|  2|Chris|  6000|     Sales| Manager|
|  3|Jamie|  7000|   Support|Director|
+---+-----+------+----------+--------+
```

## B. Spark Streaming

1. Akses Kafka seperti di modul praktikum 11

```
su kafka
cd /usr/hdp/3.0.1.0-187/kafka
```

2. Buat sebuah topic misal **json_logs**

```
bin/kafka-topics.sh --create --zookeeper localhost:2181 --
replication-factor 1 --partitions 1 --topic json_logs
```

```
[kafka@sandbox-hdp kafka]$ bin/kafka-topics.sh --create --zookeeper localhost:2181 --replication-factor 1 --partitions 1
 --topic json_logs
WARNING: Due to limitations in metric names, topics with a period ('.') or underscore ('_') could collide. To avoid issu
es it is best to use either, but not both.
Created topic "json_logs".
[kafka@sandbox-hdp kafka]$ bin/kafka-topics.sh --list --zookeeper localhost:2181
ATLAS_ENTITIES
ATLAS_HOOK
__consumer_offsets
car_gps
json_logs
tpd_logs
```

3. Masuk sebagai Producer dan publish message di bawah ini satu per satu

```
bin/kafka-console-producer.sh --broker-list sandbox-hdp.hortonworks.com:6667 --topic json_logs
```

```
{"id":1,"firstname":"James
","middlename":"","lastname":"Smith","dob_year":2018,"dob_mon
th":1,"gender":"M","salary":3000}
{"id":2,"firstname":"Michael
","middlename":"Rose","lastname":"","dob_year":2010,"dob_mont
h":3,"gender":"M","salary":4000}
{"id":3,"firstname":"Robert
","middlename":"","lastname":"Williams","dob_year":2010,"dob_
month":3,"gender":"M","salary":4000}
{"id":4,"firstname":"Maria
","middlename":"Anne","lastname":"Jones","dob_year":2005,"dob
_month":5,"gender":"F","salary":4000}
{"id":5,"firstname":"Jen","middlename":"Mary","lastname":"Bro
wn","dob_year":2010,"dob_month":7,"gender":"","salary":-1}
```

```
[kafka@sandbox-hdp kafka]$ bin/kafka-console-producer.sh --broker-list sandbox-hdp.hortonworks.com:6667 --topic json_log
s
>{"id":1,"firstname":"James ","middlename":"","lastname":"Smith","dob_year":2018,"dob_month":1,"gender":"M","salary":300
0}
>{"id":2,"firstname":"Michael ","middlename":"Rose","lastname":"","dob_year":2010,"dob_month":3,"gender":"M","salary":40
00}
>{"id":3,"firstname":"Robert ","middlename":"","lastname":"Williams","dob_year":2010,"dob_month":3,"gender":"M","salary"
:4000}
>{"id":4,"firstname":"Maria ","middlename":"Anne","lastname":"Jones","dob_year":2005,"dob_month":5,"gender":"F","salary"
:4000}
>{"id":5,"firstname":"Jen","middlename":"Mary","lastname":"Brown","dob_year":2010,"dob_month":7,"gender":"","salary":-1}
>
```

4. Pada Tab baru MobaXterm, buka spark-shell dengan menambahkan package spark-sql-kafka, dimana kita harus menyesuaikan dengan versi spark yang kita gunakan. Cari package pada:




Kita akan menggunakan package versi 0-10_2.11 untuk spark versi 2.3.1

```
spark-shell --packages org.apache.spark:spark-sql-kafka-0-10_2.11:2.3.1
```

```
[root@sandbox-hdp ~]# spark-shell --packages org.apache.spark:spark-sql-kafka-0-10_2.11:2.3.1
SPARK_MAJOR_VERSION is set to 2, using Spark2
Ivy Default Cache set to: /root/.ivy2/cache
The jars for the packages stored in: /root/.ivy2/jars
:: loading settings :: url = jar:file:/usr/hdp/3.0.1.0-187/spark2/jars/ivy-2.4.0.jar!/org/apache/ivy/
ttings.xml
org.apache.spark#spark-sql-kafka-0-10_2.11 added as a dependency
:: resolving dependencies :: org.apache.spark#spark-submit-parent-eada0c97-cb4d-4501-8ae4-44affd40c5e
        confs: [default]
        found org.apache.spark#spark-sql-kafka-0-10_2.11;2.3.1 in central
        found org.apache.kafka#kafka-clients;0.10.0.1 in central
        found net.jpountz.lz4#lz4;1.3.0 in central
        found org.xerial.snappy#snappy-java;1.1.2.6 in central
        found org.slf4j#slf4j-api;1.7.16 in central
        found org.spark-project.spark#unused;1.0.0 in central
downloading https://repo1.maven.org/maven2/org/apache/spark/spark-sql-kafka-0-10_2.11/2.3.1/spark-sql
3.1.jar ...
        [SUCCESSFUL ] org.apache.spark#spark-sql-kafka-0-10_2.11;2.3.1!spark-sql-kafka-0-10_2.11.jar
downloading https://repo1.maven.org/maven2/org/apache/kafka/kafka-clients/0.10.0.1/kafka-clients-0.10
        [SUCCESSFUL ] org.apache.kafka#kafka-clients;0.10.0.1!kafka-clients.jar (2918ms)
downloading https://repo1.maven.org/maven2/net/jpountz/lz4/lz4/1.3.0/lz4-1.3.0.jar ...
```

5. Setelah sukses masuk pada spark-shell, lakukan import libraries terlebih dahulu

```
scala> import org.apache.spark.sql.SparkSession
scala> import org.apache.spark.sql.streaming._
scala> import org.apache.spark.sql.functions._
```

```
scala> import org.apache.spark.sql.SparkSession
import org.apache.spark.sql.SparkSession

scala> import org.apache.spark.sql.streaming._
import org.apache.spark.sql.streaming._

scala> import org.apache.spark.sql.functions._
import org.apache.spark.sql.functions._
```

6. Eksekusi perintah-perintah berikut untuk membaca data (readStream) dari Kafka

```
scala> val df =
spark.readStream.format("kafka").option("kafka.bootstrap.serv
ers", "sandbox-hdp.hortonworks.com:6667").option("subscribe",
"json_logs").option("startingOffsets", "earliest").load()
scala> df.printSchema()
```

```
scala> val df = spark.readStream.format("kafka").option("kafka.bootstrap.servers", "sandbox-hdp.hortonworks.com:6667").o
ption("subscribe", "json_logs").option("startingOffsets", "earliest").load()
df: org.apache.spark.sql.DataFrame = [key: binary, value: binary ... 5 more fields]

scala> df.printSchema()
root
 |-- key: binary (nullable = true)
 |-- value: binary (nullable = true)
 |-- topic: string (nullable = true)
 |-- partition: integer (nullable = true)
 |-- offset: long (nullable = true)
 |-- timestamp: timestamp (nullable = true)
 |-- timestampType: integer (nullable = true)
```

7. Nantinya setiap message baru akan tersimpan pada df. Misalkan kita ingin menampilkan dalam bentuk tabel dengan skema tertentu, gunakan perintah berikut.

```
scala> val personStringDF = df.selectExpr("CAST(value AS
STRING)")
scala> import org.apache.spark.sql.types._
scala> val kafkaSchema = StructType(Seq(StructField("id",
IntegerType), StructField("firstname", StringType),
StructField("middlename", StringType),
StructField("lastname", StringType), StructField("dob_year",
IntegerType), StructField("dob_month", IntegerType),
StructField("gender", StringType), StructField("salary",
IntegerType)))
```

```
scala> val personDF =
personStringDF.select(from_json(col("value"),
kafkaSchema).as("data")).select("data.*")
scala>
personDF.writeStream.format("console").outputMode("append").s
tart().awaitTermination()
```

```
scala> import org.apache.spark.sql.types._
import org.apache.spark.sql.types._

scala> val kafkaSchema = StructType(Seq(StructField("id", IntegerType), StructField("firstname", StringType), StructFiel
d("middlename", StringType), StructField("lastname", StringType), StructField("dob_year", IntegerType), StructField("dob
_month", IntegerType), StructField("gender", StringType), StructField("salary", IntegerType)))
kafkaSchema: org.apache.spark.sql.types.StructType = StructType(StructField(id,IntegerType,true), StructField(firstname,
StringType,true), StructField(middlename,StringType,true), StructField(lastname,StringType,true), StructField(dob_year,I
ntegerType,true), StructField(dob_month,IntegerType,true), StructField(gender,StringType,true), StructField(salary,Integ
erType,true))

scala> val personDF = personStringDF.select(from_json(col("value"), kafkaSchema).as("data")).select("data.*")
personDF: org.apache.spark.sql.DataFrame = [id: int, firstname: string ... 6 more fields]

scala> personDF.writeStream.format("console").outputMode("append").start().awaitTermination()
-------------------------------------------
Batch: 0
-------------------------------------------
+---+---------+----------+--------+--------+---------+------+------+
| id|firstname|middlename|lastname|dob_year|dob_month|gender|salary|
+---+---------+----------+--------+--------+---------+------+------+
|  1|   James |          |   Smith|    2018|        1|     M|  3000|
|  2| Michael |      Rose|        |    2010|        3|     M|  4000|
|  3|  Robert |          |Williams|    2010|        3|     M|  4000|
|  4|   Maria |      Anne|   Jones|    2005|        5|     F|  4000|
|  5|      Jen|      Mary|   Brown|    2010|        7|      |    -1|
+---+---------+----------+--------+--------+---------+------+------+
```

8. Setiap message baru masuk, misal kita menambahkan message person baru, maka akan terbentuk batch data baru pada **personDF**.

```
{"id":6,"firstname":"Tony,"lastname":"Brown","dob_year":2011,
"dob_month":6,"gender":"M","salary":3000}
```

```
[kafka@sandbox-hdp kafka]$ bin/kafka-console-producer.sh --broker-list sandbox-hdp.hortonworks.com:6667 --topic json_log
s
>{"id":6,"firstname":"Tony","lastname":"Brown","dob_year":2011,"dob_month":6,"gender":"M","salary":3000}
>
```

```
-------------------------------------------
Batch: 0
-------------------------------------------
+---+---------+----------+--------+--------+---------+------+------+
| id|firstname|middlename|lastname|dob_year|dob_month|gender|salary|
+---+---------+----------+--------+--------+---------+------+------+
|  1|   James |          |   Smith|    2018|        1|     M|  3000|
|  2| Michael |      Rose|        |    2010|        3|     M|  4000|
|  3|  Robert |          |Williams|    2010|        3|     M|  4000|
|  4|   Maria |      Anne|   Jones|    2005|        5|     F|  4000|
|  5|      Jen|      Mary|   Brown|    2010|        7|      |    -1|
+---+---------+----------+--------+--------+---------+------+------+

-------------------------------------------
Batch: 1
-------------------------------------------
+---+---------+----------+--------+--------+---------+------+------+
| id|firstname|middlename|lastname|dob_year|dob_month|gender|salary|
+---+---------+----------+--------+--------+---------+------+------+
|  6|     Tony|      null|   Brown|    2011|        6|     M|  3000|
+---+---------+----------+--------+--------+---------+------+------+
```

Data siap untuk diproses lebih lanjut dari **personDF**.

### 1.5. Penugasan

Berikut adalah data yang akan dijadikan case:

```
'James','','Smith','1991-04-01','M',3000
'Michael','Rose','','2000-05-19','M',4000
'Robert','','Williams','1978-09-05','M',4000
'Maria','Anne','Jones','1967-12-01','F',4000
'Jen','Mary','Brown','1980-02-17','F',10000
```

Skema yang digunakan:

```
"firstname","middlename","lastname","dob","gender","salary"
```

1. Lakukan input data dalam bentuk RDD.
2. Mapping dengan skema yang telah disiapkan dan simpan dalam bentuk dataframe.
3. Lakukan operasi terhadap dataframe yaitu:
   - Filter record dengan tahun lahir > 2000
   - Group by Gender
   - Bentuk kolom baru yang berisi gabungan antara first name, middle name, dan last name
4. Penggunaan Spark SQL
   - Simpan kolom baru yang berisi gabungan antara first name, middle name, dan last name, dob, gender, dan salary ke dalam tabel hive.
   - Lakukan transformasi dengan menambah kolom umur yang berisi hasil pengurangan tahun 2023 dengan tahun lahir
   - Tampilkan hasil transformasi
5. Simpan hasil transformasi pada nomor 4 dalam format orc di HDFS